

BÁO CÁO KIẾN TRÚC TỔNG THỂ

Hệ thống chấm bài tự động — AutoGrade

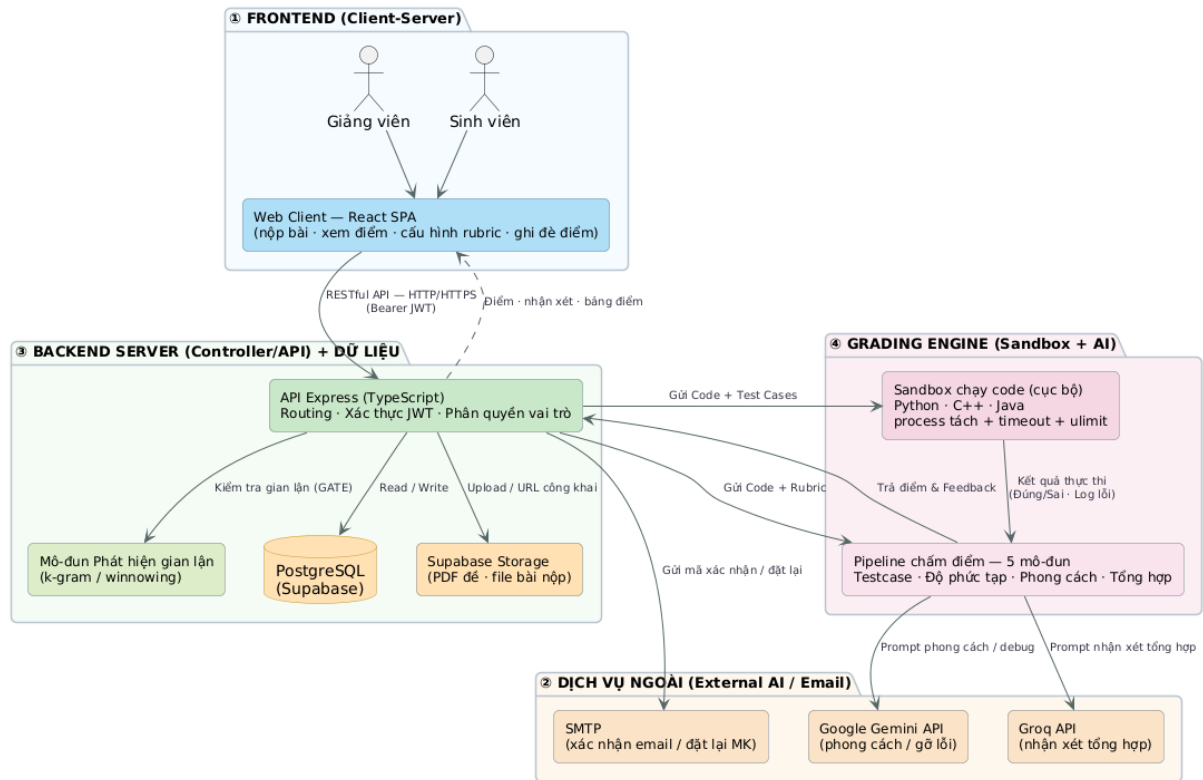
Đồ án môn Công nghệ phần mềm cho AI

Trường Đại học Khoa học Tự nhiên, ĐHQG-HCM (HCMUS)

Mục lục

Sơ đồ kiến trúc tổng thể	3
1. Tổng quan kiến trúc	3
2. Phân tích các phân hệ (Layers)	4
2.1 Lớp Frontend (Client–Server)	4
2.2 Lớp Dịch vụ ngoài (External Services).....	4
2.3 Lớp Backend Server (Controller/API) + Dữ liệu	4
2.4 Lớp Grading Engine (Sandbox + AI)	4
3. Các luồng chấm điểm (Grading Flows)	5
3.1 Luồng A — Chấm hàng loạt bài tập lớp	5
3.2 Luồng B — Chấm tức thời bài tự luyện	5
3.3 Phần lỗi pipeline (rẽ nhánh bên trong).....	5
3.4 Luồng “Chạy thử” (không lưu, không tính điểm)	6
4. Luồng dữ liệu tổng quát	7
5. Công nghệ chính (tóm tắt)	8

Sơ đồ kiến trúc tổng thể



Hình 1. Kiến trúc tổng thể AutoGrade — 4 vùng và các luồng dữ liệu chính.

1. Tổng quan kiến trúc

Hệ thống theo mô hình Client–Server nhiều tầng, tách rời Frontend và Backend, giao tiếp bằng RESTful API (HTTP/HTTPS) kèm JWT. Toàn bộ được chia thành 4 vùng (phân hệ) độc lập để dễ bảo trì và mở rộng:

- **① Frontend (Client–Server):** giao diện web SPA cho Sinh viên và Giảng viên.
- **② Dịch vụ ngoài:** các dịch vụ AI (Gemini, Groq) và gửi email (SMTP).
- **③ Backend Server (Controller/API) + Dữ liệu:** điều phối nghiệp vụ, xác thực, và lưu trữ bền vững (PostgreSQL/Storage của Supabase).
- **④ Grading Engine (Sandbox + AI):** môi trường chạy code cách ly và pipeline chấm điểm 5 mô-đun.

Khác với kiến trúc đa-agent tổng quát, phân hệ AI ở đây là một pipeline cố định 5 mô-đun (không phải hệ nhiều agent tự điều phối); sandbox chạy tiến trình cục bộ (không dùng Docker); và không tích hợp GitHub — Sinh viên nộp mã trực tiếp trên web. Điểm số luôn được tính tất định từ testcase/độ phức tạp, còn AI chỉ là lớp tăng cường nhận xét.

2. Phân tích các phân hệ (Layers)

2.1 Lớp Frontend (Client–Server)

Giao diện web SPA xây bằng ReactJS + Vite + TypeScript, giao tiếp trực tiếp với người dùng cuối:

- **Sinh viên:** nộp bài (dán code hoặc tải file), chạy thử, xem điểm và nhận xét.
- **Giảng viên:** tạo lớp/bài tập, cấu hình Rubric, kích hoạt chấm tự động, ghi đề điểm thủ công, công bố điểm, xuất bảng điểm ra Excel (phía client).
- **Giao tiếp:** gọi Backend qua RESTful API (HTTP/HTTPS); mọi yêu cầu đính kèm token JWT trong header Authorization.

2.2 Lớp Dịch vụ ngoài (External Services)

Backend gọi ra các dịch vụ bên thứ ba; tất cả đều tùy chọn và có cơ chế suy biến an toàn:

- **Google Gemini API:** nhận xét phong cách code (review) và gỡ lỗi (debug) khi bài không chạy được.
- **Groq API:** viết nhận xét tổng hợp ở bước tổng kết điểm.
- **SMTP (Brevo):** gửi email xác nhận tài khoản và đặt lại mật khẩu, thông qua SMTP relay của Brevo (chế độ DEV in mã ra console).

```
SMTP_HOST=smtp-relay.brevo.com
SMTP_PORT=587
SMTP_USER=xxx
SMTP_PASS=xxx
SMTP_FROM="AutoGrade <kpuithuong@gmail.com>"
```

2.3 Lớp Backend Server (Controller/API) + Dữ liệu

“Trạm trung chuyển” xử lý logic nghiệp vụ, dùng Node.js + Express + TypeScript:

- **API Gateway / Controller (Express):** tiếp nhận mọi yêu cầu từ Frontend, xác thực JWT, phân quyền theo vai trò (Sinh viên/Giảng viên), điều phối tới các mô-đun khác.
- **Database (PostgreSQL — Supabase):** lưu trữ bền vững người dùng, lớp, bài tập, bài nộp, điểm, tiến trình chấm. Backend là nơi duy nhất chạm DB (service_role key) nên không cần bật RLS.
- **Supabase Storage:** lưu file (PDF đề, file bài nộp) và trả URL công khai, bền vững qua các lần triển khai.
- **Mô-đun Phát hiện gian lận:** so khớp k-gram/winnowing giữa các bài nộp; đóng vai trò cổng chặn (GATE) trong luồng chấm bài nộp.

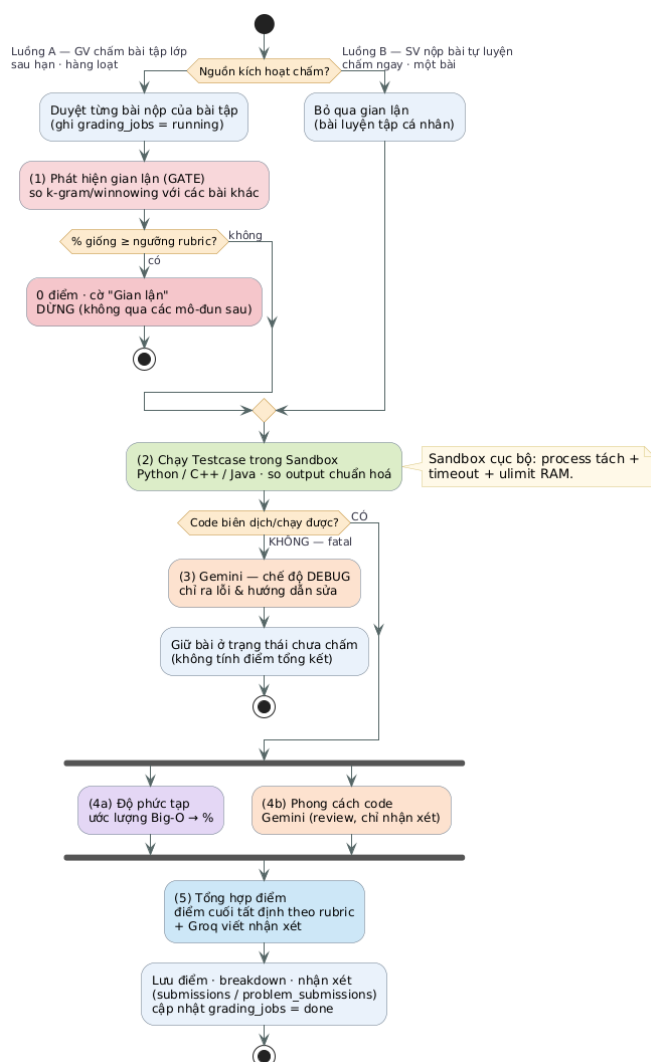
2.4 Lớp Grading Engine (Sandbox + AI)

Phân hệ xử lý lỗi, gồm môi trường chạy code cách ly và pipeline chấm điểm:

- **Sandbox Engine (cục bộ):** biên dịch và chạy thử mã nguồn với Test Cases một cách an toàn (process tách + timeout + giới hạn RAM), hỗ trợ Python/C++/Java trên Windows/macOS/Linux. Kết quả (Đúng/Sai, Log lỗi) được đẩy vào pipeline.
- **Pipeline chấm điểm — 5 mô-đun:** Phát hiện gian lận (GATE) · Testcase · Độ phức tạp (Big-O) · Phong cách code (Gemini) · Tổng hợp điểm & nhận xét (Groq). Trình tự có rẽ nhánh (xem Mục 3).

3. Các luồng chấm điểm (Grading Flows)

Hệ thống có hai luồng kích hoạt chấm khác nhau nhưng dùng chung phần lõi pipeline. Điểm khác biệt chính là bước Phát hiện gian lận chỉ áp dụng cho bài tập lớp.



Hình 2. Hai luồng chấm điểm (A: bài tập lớp · B: bài tự luyện) và pipeline dùng chung.

3.1 Luồng A — Chấm hàng loạt bài tập lớp

Do Giảng viên kích hoạt sau hạn nộp; hệ thống duyệt từng bài nộp của bài tập:

- **Bước 1 — Gian lận (GATE):** so k-gram/winnowing bài đang xét với các bài còn lại; nếu % giống $\geq$ ngưỡng rubric $\rightarrow$ 0 điểm, gán cò “Gian lận” và DỪNG (không qua các mô-đun sau).
- **Bước 2 $\rightarrow$ 5:** nếu không gian lận, đi tiếp phần lõi pipeline (Testcase $\rightarrow$ rẽ nhánh $\rightarrow$ Độ phức tạp || Phong cách $\rightarrow$ Tổng hợp).

3.2 Luồng B — Chấm tức thời bài tự luyện

Kích hoạt ngay khi Sinh viên nộp một bài tự luyện; bỏ qua bước gian lận (bài luyện tập cá nhân), rồi chạy phần lõi pipeline như Luồng A và trả điểm/nhận xét ngay lập tức. Mỗi lần nộp lưu thành một bản ghi riêng để giữ lịch sử.

3.3 Phần lõi pipeline (rẽ nhánh bên trong)

- **Testcase (Sandbox):** biên dịch & chạy, so output đã chuẩn hoá → % testcase đạt.
- **Nếu code KHÔNG chạy được (fatal):** chuyển Gemini sang chế độ debug để chỉ lỗi & hướng dẫn sửa, KHÔNG tính điểm tổng kết và dừng bài đó.
- **Nếu code chạy được:** chạy song song Độ phức tạp (Big-O → %) và Phong cách code (Gemini review, chỉ nhận xét).
- **Tổng hợp:** tính điểm cuối tất định theo trọng số rubric, gọi Groq viết nhận xét, rồi lưu điểm + breakdown + nhận xét và đóng grading_job (done).

Ghi chú: nếu thiếu API key Gemini/Groq hoặc dịch vụ lỗi, hệ thống dùng nhận xét dự phòng ghép từ số liệu; điểm số vẫn tính bình thường vì không phụ thuộc kết quả AI.

3.4 Luồng “Chạy thử” (không lưu, không tính điểm)

Trước khi nộp, Sinh viên có thể “Chạy thử” code với các testcase công khai để xem kết quả tức thời. Luồng này chỉ dùng Sandbox, không ghi bài nộp và không tạo grading_job.

4. Luồng dữ liệu tổng quát

Trình tự tổng quát từ lúc người dùng thao tác đến khi có điểm:

1. Sinh viên/Giảng viên thao tác trên Web Client; mọi yêu cầu gọi Backend qua RESTful API (HTTP/HTTPS) kèm Bearer JWT.
2. API Gateway (Express) xác thực JWT, phân quyền vai trò, rồi đọc/ghi dữ liệu tại PostgreSQL (và Storage cho file).
3. Khi nộp bài, hệ thống lưu bản ghi bài nộp; bài tập lớp ở trạng thái “submitted” chờ Giảng viên chấm, bài tự luyện được chấm ngay.
4. Khi kích hoạt chấm, service tạo `grading_job` (running) và đưa dữ liệu vào Grading Engine.
5. Pipeline chạy: Gian lận (GATE, chỉ bài lớp) → Sandbox chạy Testcase → (rẽ nhánh lỗi: Gemini debug) → Độ phức tạp // Phong cách (Gemini) → Tổng hợp (Grok).
6. Kết quả (điểm chức năng từ Sandbox + chất lượng/nhận xét từ AI) được tổng hợp thành điểm cuối và Feedback.
7. Backend lưu điểm/nhận xét vào Database, đóng `grading_job` (done), và trả kết quả để hiển thị trên Frontend; Giảng viên rà soát, chỉnh, công bố; bảng điểm có thể xuất Excel.

5. Công nghệ chính (tóm tắt)

Phân hệ	Công nghệ chính
Frontend	React 18 · Vite · TypeScript · React Router · Zustand · TailwindCSS + shadcn/ui · Monaco Editor · SheetJS (xlsx)
Backend/API	Node.js · Express · TypeScript · JWT (jsonwebtoken) · bcryptjs · Multer · Nodemailer · cors/dotenv
Dữ liệu & Hạ tầng	PostgreSQL (Supabase) · Supabase Storage · @supabase/supabase-js (service_role)
Grading Engine	Sandbox cục bộ (Python/C++/Java) · pipeline 5 mô-đun · Gemini API (phong cách/debug) · Groq API (tổng hợp)

Chi tiết đầy đủ về phân tầng, API, mô hình dữ liệu và phiên bản công nghệ được trình bày trong tài liệu “Kiến trúc hệ thống & Công nghệ sử dụng”.